Mining Frequent Patterns, Association and Correlations

What Is Frequent Pattern Analysis?

- Frequent pattern: a pattern (a set of items, subsequences, substructures, etc.) that occurs frequently in a data set
- First proposed by Agrawal, Imielinski, and Swami [AIS93] in the context of frequent itemsets and association rule mining
- Motivation: Finding inherent regularities in data
  - What products were often purchased together?— Beer and diapers?!
  - What are the subsequent purchases after buying a PC?
  - What kinds of DNA are sensitive to this new drug?
  - Can we automatically classify web documents?
- Applications
  - Basket data analysis, cross-marketing, catalog design, sale campaign analysis, Web log (click stream) analysis, and DNA sequence analysis.

Why Is Freq. Pattern Mining Important?

- Freq. pattern: An intrinsic and important property of datasets
- Foundation for many essential data mining tasks
  - Association, correlation, and causality analysis
  - Sequential, structural (e.g., sub-graph) patterns
  - Pattern analysis in spatiotemporal, multimedia, time-series, and stream data
  - Classification: discriminative, frequent pattern analysis
  - Cluster analysis: frequent pattern-based clustering
  - Data warehousing: iceberg cube and cube-gradient
  - Semantic data compression: fascicles
  - Broad applications

The model: data

- $I = \{i_1, i_2, \ldots, i_m\}$: a set of *items*.
- Transaction t :
  - t a set of items, and $t \subseteq I$.
- Transaction Database T: a set of transactions $T = \{t_1, t_2, \ldots, t_n\}$.

Transaction data: supermarket data

- Market basket transactions:

  t1: {bread, cheese, milk}

  t2: {apple, eggs, salt, yogurt}

  … …

  tn: {biscuit, eggs, milk}

- Concepts:
  - An *item*: an item/article in a basket
  - *I*: the set of all items sold in the store
  - A *transaction*: items purchased in a basket; it may have TID (transaction ID)
  - A *transactional dataset*: A set of transactions

Transaction data: a set of documents

- **A text document data set. Each document is treated as a "bag" of keywords**

doc1:	Student, Teach, School
doc2:	Student, School
doc3:	Teach, School, City, Game
doc4:	Baseball, Basketball
doc5:	Basketball, Player, Spectator
doc6:	Baseball, Coach, Game, Team
doc7:	Basketball, Team, City, Game

The model: rules

- A transaction *t* contains *X*, a set of items (itemset) in *I*, if $X \subseteq t$.
- An association rule is an implication of the form:

  $X \rightarrow Y$, where $X, Y \subset I$, *and* $X \cap Y = \varnothing$

- An itemset is a set of items.
  - E.g., X = {milk, bread, cereal} is an itemset.
- A *k*-itemset is an itemset with *k* items.
  - E.g., {milk, bread, cereal} is a 3-itemset

Rule strength measures

- Support: The rule holds with support *sup* in *T* (the transaction data set) if sup% of transactions contain $X \cup Y$.
  - $sup = \Pr(X \cup Y)$.
- Confidence: The rule holds in *T* with confidence *conf* if *conf*% of tranactions that contain *X* also contain *Y.*
  - $conf = \Pr(Y \mid X)$
- An association rule is a pattern that states when *X* occurs, *Y* occurs with certain probability.

Support and Confidence

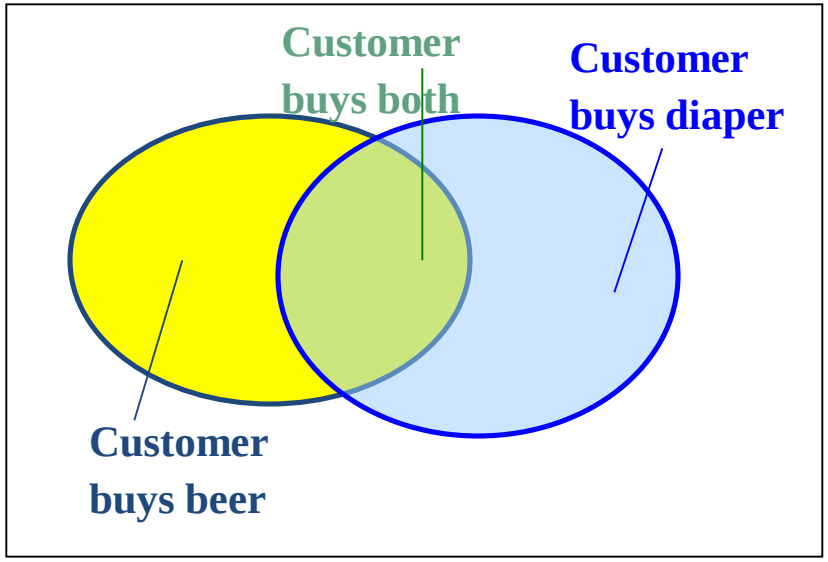
- Support count: The support count of an itemset *X*, denoted by *X.count*, in a data set *T* is the number of transactions in *T* that contain *X*. Assume *T* has *n* transactions.
- Then,

$$support = \frac{(X \cup Y).count}{n}$$

$$confidence = \frac{(X \cup Y).count}{X.count}$$

Basic Concepts: Frequent Patterns

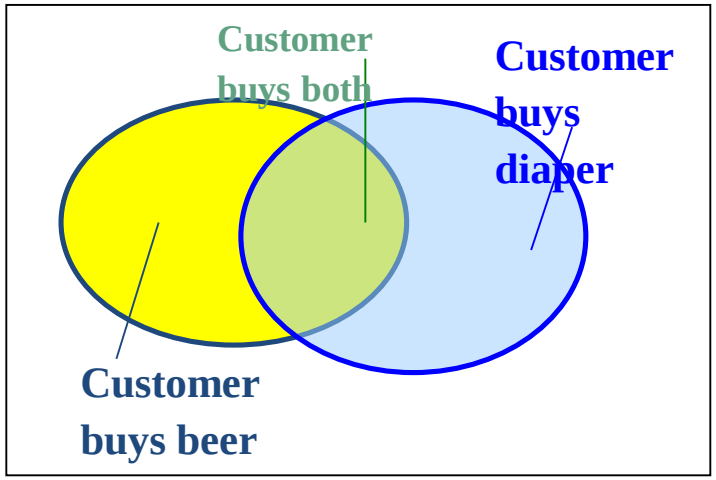
Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



- itemset: A set of one or more items
- k-itemset X = $\{x_1, \ldots, x_k\}$
- *(absolute) support*, or, *support count* of X: Frequency or occurrence of an itemset X
- *(relative) support*, *s*, is the fraction of transactions that contains X (i.e., the probability that a transaction contains X)
- An itemset X is *frequent* if X's support is no less than a *minsup* threshold

Basic Concepts: Association Rules

Tid	Items bought
10	Beer, Nuts, Diaper
20	Beer, Coffee, Diaper
30	Beer, Diaper, Eggs
40	Nuts, Eggs, Milk
50	Nuts, Coffee, Diaper, Eggs, Milk



- Find all the rules $X \rightarrow Y$ with minimum support and confidence
  - support, *s*, probability that a transaction contains $X \cup Y$
  - confidence, *c*, conditional probability that a transaction having X also contains *Y*

Let minsup = 50%, minconf = 50%

Freq. Pat.: Beer:3, Nuts:3, Diaper:4, Eggs:3, {Beer, Diaper}:3

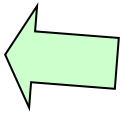
- Association rules: (many more!)
  - *Beer* → *Diaper* (60%, 100%)
  - *Diaper* → *Beer* (60%, 75%)

Computational Complexity of Frequent Itemset Mining

- How many itemsets are potentially to be generated in the worst case?
  - The number of frequent itemsets to be generated is senstive to the minsup threshold
  - When minsup is low, there exist potentially an exponential number of frequent itemsets
  - The worst case: M^N where M: # distinct items, and N: max length of transactions
- The worst case complexty vs. the expected probability
  - Ex. Suppose Walmart has 10^4 kinds of products
    - The chance to pick up one product 10^{-4}
    - The chance to pick up a particular set of 10 products: ~10^{-40}
    - What is the chance this particular set of 10 products to be frequent 10^3 times in 10^9 transactions?

Scalable Frequent Itemset Mining Methods

- Apriori: A Candidate Generation-and-Test Approach



- Improving the Efficiency of Apriori
- FPGrowth: A Frequent Pattern-Growth Approach
- ECLAT: Frequent Pattern Mining with Vertical Data Format

The Downward Closure Property and Scalable Mining Methods

- The downward closure property of frequent patterns
  - Any subset of a frequent itemset must be frequent
  - If **{beer, diaper, nuts}** is frequent, so is **{beer, diaper}**
  - i.e., every transaction having {beer, diaper, nuts} also contains {beer, diaper}
- Scalable mining methods: Three major approaches
  - Apriori
  - Freq. pattern growth
  - Vertical data format approach

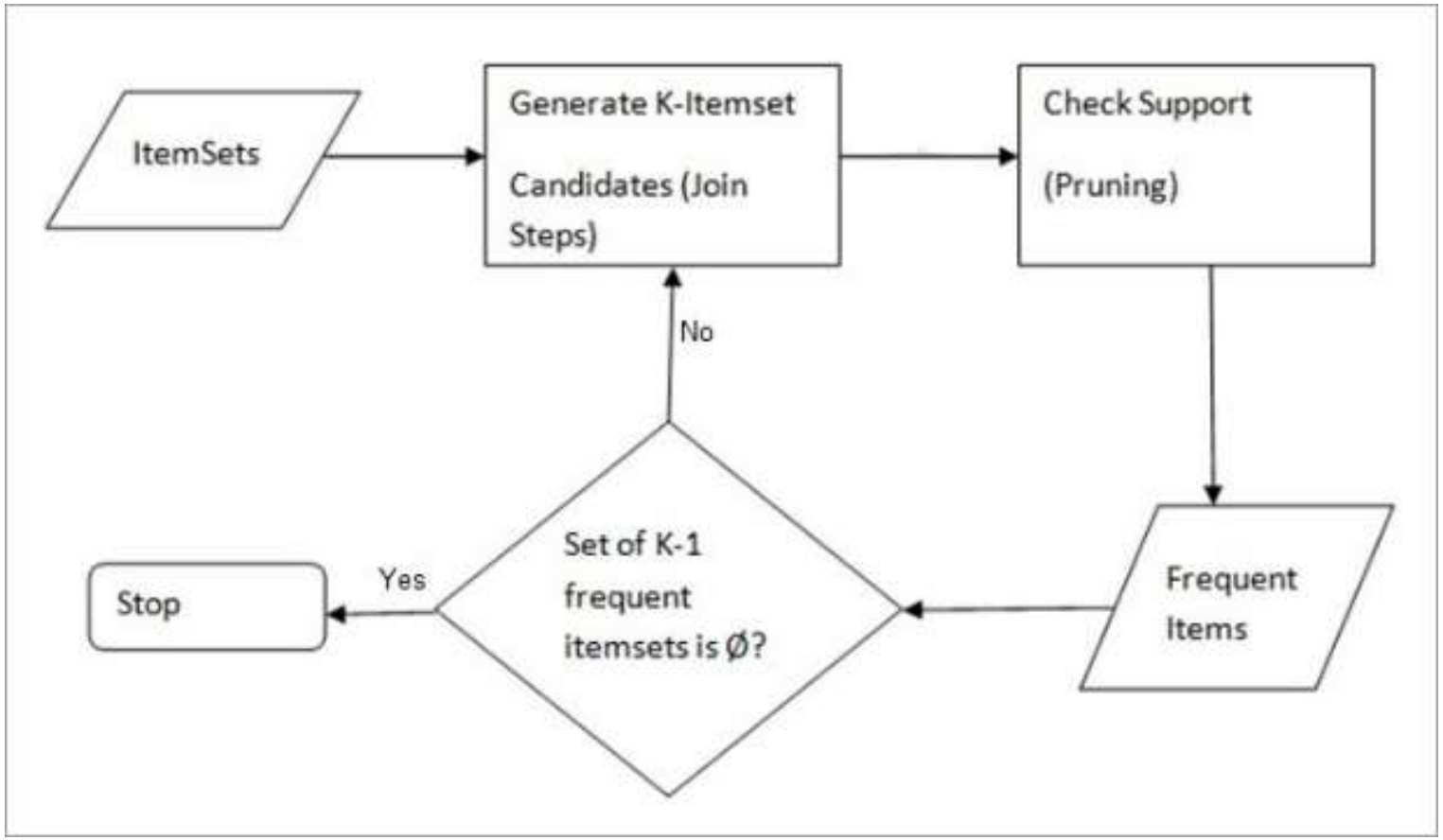
Apriori: A Candidate Generation & Test Approach

- Apriori pruning principle: If there is any itemset which is infrequent, its superset should not be generated/tested!

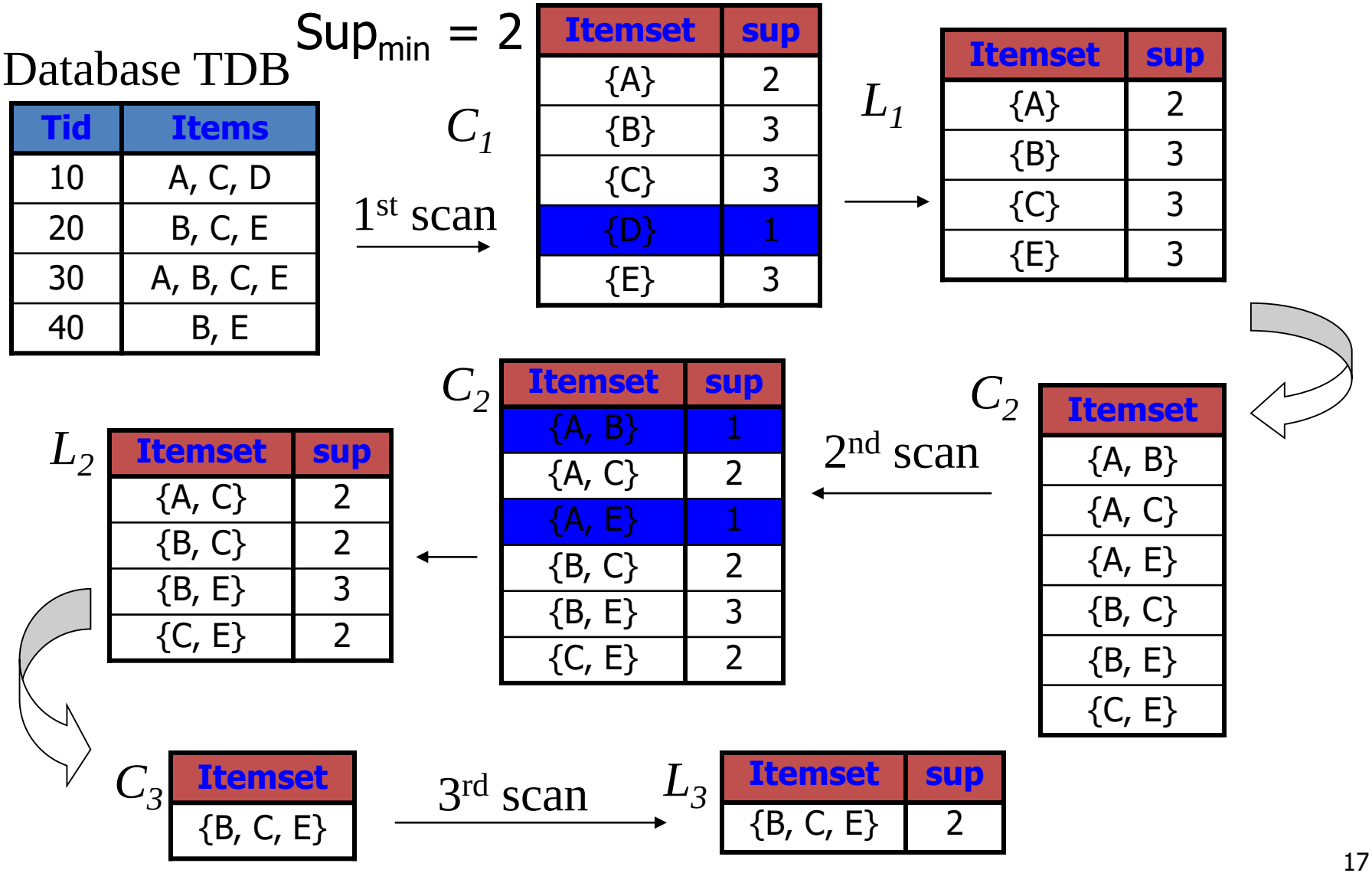
  Method:

  - Initially, scan DB once to get frequent 1-itemset
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
  - Test the candidates against DB
  - Terminate when no frequent or candidate set can be generated

Steps in Apriori Algorithm



The Apriori Algorithm—An Example



The Apriori Algorithm (Pseudo-Code)

C_k: Candidate itemset of size k

L_k : frequent itemset of size k

L_1 = {frequent items};

for (k = 1; L_k !=∅; k++) **do begin**

  C_{k+1} = candidates generated from L_k;

  for each transaction t in database do

    increment the count of all candidates in C_{k+1} that are contained in t

  L_{k+1} = candidates in C_{k+1} with min_support

  end

return $\cup_k L_k$;

Implementation of Apriori

- How to generate candidates?
  - Step 1: self-joining L_k
  - Step 2: pruning
- Example of Candidate-generation
  - L_3={*abc, abd, acd, ace, bcd*}
  - Self-joining: $L_3 * L_3$
    - *abcd* from *abc* and *abd*
    - *acde* from *acd* and *ace*
  - Pruning:
    - *acde* is removed because *ade* is not in L_3
  - C_4 = {*abcd*}

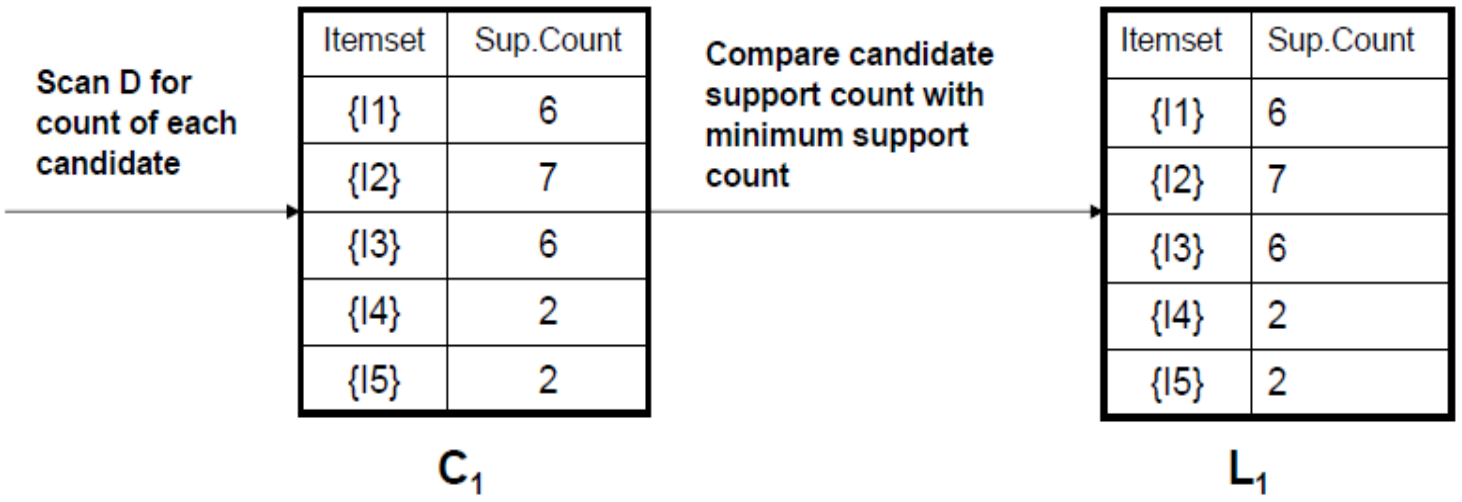
Apriori Example

- A Marketing manager of All-Electronics company branch would like to learn more about the buying habits of the customers to plan marketing strategies. Specifically he is interested to know “Which groups of items are customers likely to purchase on a given trip to the store?”. Consider the All-Electronics transaction database.

TID	List of item ID's
100	I1, I2,I5
101	I2,I4
102	I2,I3
103	I1,I2,I4
104	I1,I3
105	I2,I3
106	I1,I3
107	I1,I2,I3,I5
108	I1,I2,I3

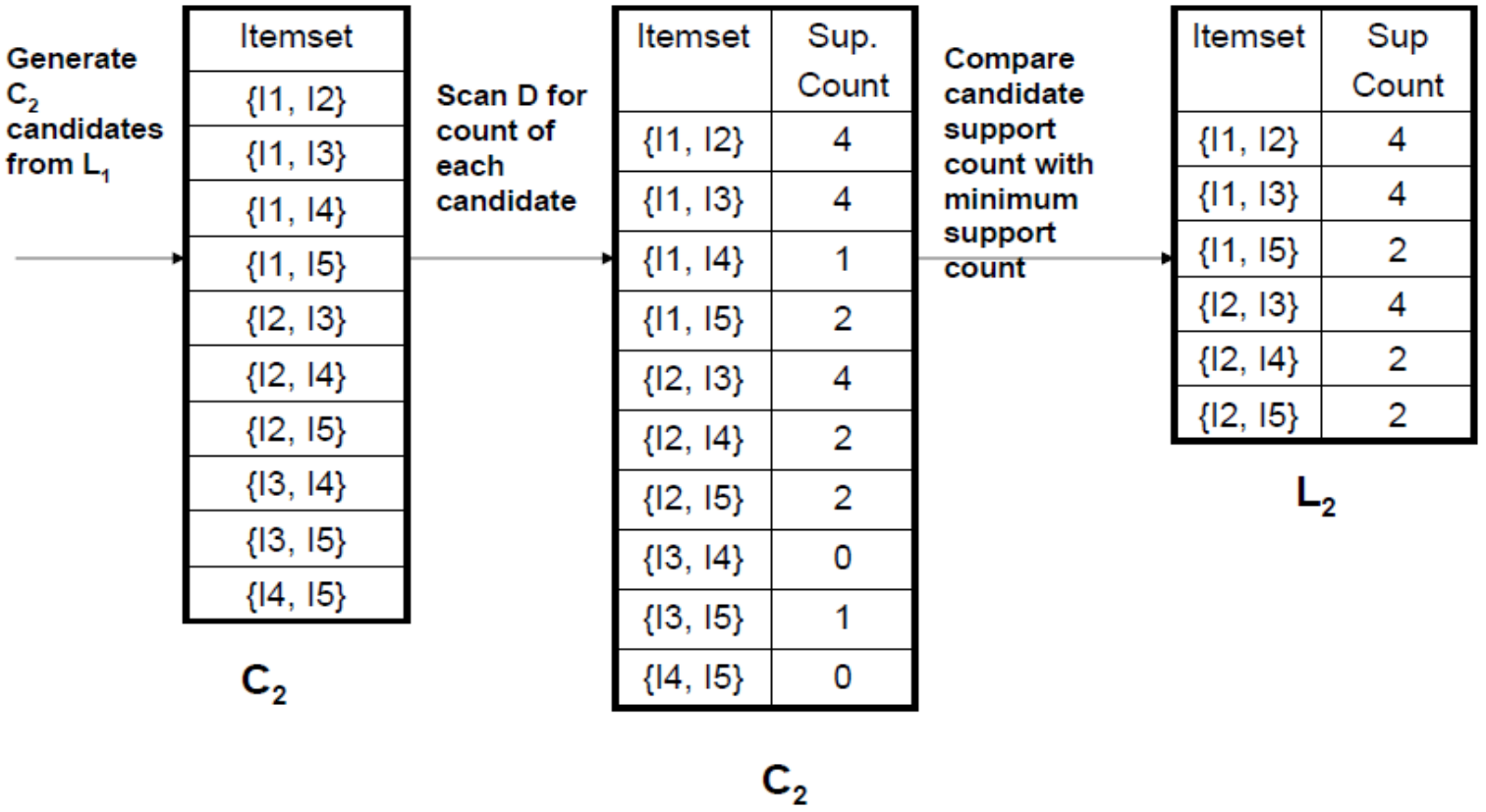
- Implement a algorithm to mine items frequently purchased together by customer with minimum support threshold for a item (or set of items) is 22% (2/9) i.e. (**min support = 2**) & minimum confidence required is **70%.**

Step 1: Generating 1-itemset Frequent Pattern



- The set of frequent 1-itemsets, L_1, consists of the candidate 1-itemsets satisfying minimum support.
- In the first iteration of the algorithm, each item is a member of the set of candidate.

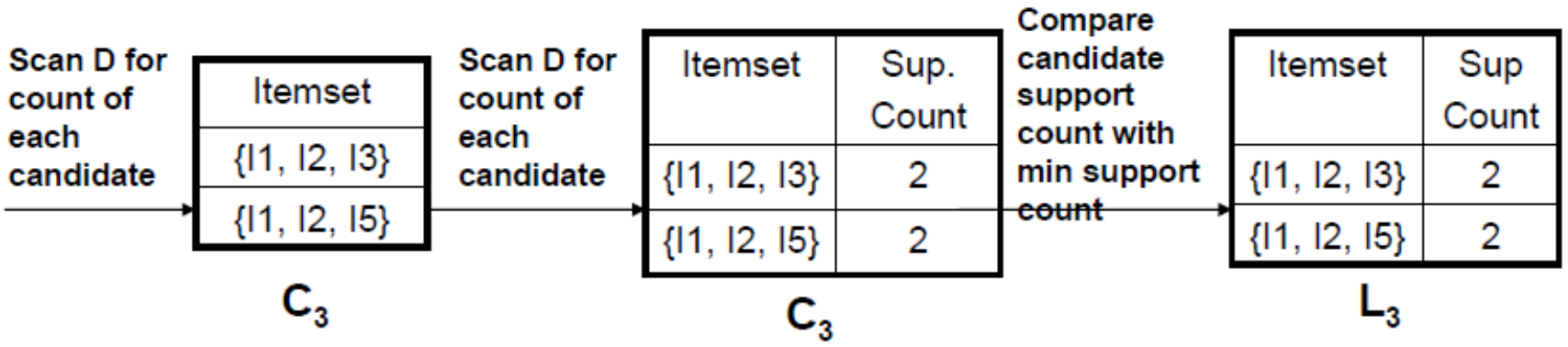
Step 2: Generating 2-itemset Frequent Pattern



Step 2: Generating 2-itemset Frequent Pattern

- To discover the set of frequent 2-itemsets, L_2 , the algorithm uses L_1 *Join* L_1 to generate a candidate set of 2-itemsets, C_2.
- Next, the transactions in D are scanned and the support count for each candidate itemset in C_2 is accumulated (as shown in the middle table).
- The set of frequent 2-itemsets, L_2 , is then determined, consisting of those candidate 2-itemsets in C_2 having minimum support.
- Note: We haven't used Apriori Property yet.

Step 3: Generating 3-itemset Frequent Pattern



- The generation of the set of candidate 3-itemsets, C_3 , involves use of the Apriori Property.
- In order to find C_3, we compute L_2 *Join* L_2.
- C_3 = L2 *Join* L2 = {{I1, I2, I3}, {I1, I2, I5}, {I1, I3, I5}, {I2, I3, I4}, {I2, I3, I5}, {I2, I4, I5}}.
- Now, Join step is complete and Prune step will be used to reduce the size of C_3. Prune step helps to avoid heavy computation due to large C_k.

Step 3: Generating 3-itemset Frequent Pattern

- Based on the Apriori property that all subsets of a frequent itemset must also be frequent, we can determine that four latter candidates cannot possibly be frequent. How ?
- For example , lets take {I1, I2, I3}. The 2-item subsets of it are {I1, I2}, {I1, I3} & {I2, I3}. Since all 2-item subsets of {I1, I2, I3} are members of L_2, We will keep {I1, I2, I3} in C_3.
- Lets take another example of {I2, I3, I5} which shows how the pruning is performed. The 2-item subsets are {I2, I3}, {I2, I5} & {I3,I5}.
- BUT, {I3, I5} is not a member of L_2 and hence it is not frequent violating Apriori Property. Thus We will have to remove {I2, I3, I5} from C_3.
- Therefore, C_3 = {{I1, I2, I3}, {I1, I2, I5}} after checking for all members of result of Join operation for Pruning.
- Now, the transactions in D are scanned in order to determine L_3, consisting of those candidates 3-itemsets in C_3 having minimum support.

Step 4: Generating 4-itemset Frequent Pattern

- The algorithm uses L_3 *Join* L_3 to generate a candidate set of 4-itemsets, C_4. Although the join results in {{I1, I2, I3, I5}}, this itemset is pruned since its subset {{I2, I3, I5}} is not frequent.
- Thus, $C_4 = \varphi$, and algorithm terminates, having found all of the frequent items. This completes our Apriori Algorithm.
- What's Next ?

  These frequent itemsets will be used to generate strong association rules (where strong association rules satisfy both minimum support & minimum confidence).

Step 5: Generating Association Rules from Frequent Itemsets

- Procedure:
  - For each frequent itemset ***"l"***, generate all nonempty subsets of ***l***.
  - For every nonempty subset ***s*** of ***l***, output the rule "**s → (l-s)**" if **support_count(l) / support_count(s) >= min_conf** where min_conf is minimum confidence threshold.

- Back To Example:

  We had L = {{I1}, {I2}, {I3}, {I4}, {I5}, {I1,I2}, {I1,I3}, {I1,I5}, {I2,I3}, {I2,I4}, {I2,I5}, {I1,I2,I3}, {I1,I2,I5}}.
  - Lets take ***l*** = {I1,I2,I5}.
  - Its all nonempty subsets are {I1,I2}, {I1,I5}, {I2,I5}, {I1}, {I2}, {I5}.

Step 5: Generating Association Rules from Frequent Itemsets

- Let minimum confidence threshold is , say 70%.
- The resulting association rules are shown below, each listed with its confidence.
  - R1: I1 ^ I2 → I5
    - Confidence = sc{I1,I2,I5}/sc{I1,I2} = 2/4 = 50%
    - R1 is Rejected.
  - R2: I1 ^ I5 → I2
    - Confidence = sc{I1,I2,I5}/sc{I1,I5} = 2/2 = 100%
    - R2 is Selected.
  - R3: I2 ^ I5 → I1
    - Confidence = sc{I1,I2,I5}/sc{I2,I5} = 2/2 = 100%
    - R3 is Selected.

Step 5: Generating Association Rules from Frequent Itemsets

- R4: I1 → I2 ^ I5
  - Confidence = sc{I1,I2,I5}/sc{I1} = 2/6 = 33%
  - R4 is Rejected.
- R5: I2 → I1 ^ I5
  - Confidence = sc{I1,I2,I5}/{I2} = 2/7 = 29%
  - R5 is Rejected.
- R6: I5 → I1 ^ I2
  - Confidence = sc{I1,I2,I5}/ {I5} = 2/2 = 100%
  - R6 is Selected.

  In this way, We have found three strong association rules.

Improving the Efficiency of Apriori

- **Major computational challenges**
  - → Huge number of candidates
  - → Multiple scans of transaction database
  - → Tedious workload of support counting for candidates
- **Improving Apriori: general ideas**
  - → Shrink number of candidates
  - → Reduce passes of transaction database scans
  - → Facilitate support counting of candidates

FP-growth: Frequent Pattern-Growth

- Adopts a divide and conquer strategy
- Compress the database representing frequent items into a **frequent –pattern tree** or **FP-tree**
  - → Retains the itemset association information
- Divid the compressed database into a set of conditional databases, each associated with one frequent item
- Mine each such databases separately

Example: FP-growth

- The first scan of data is the same as Apriori
- Derive the set of frequent 1-itemsets
- Let min-sup=2
- Generate a set of ordered items

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2

Transactional Database

TID	List of item IDS
T100	I1,I2,I5
T200	I2,I4
T300	I2,I3
T400	I1,I2,I4
T500	I1,I3
T600	I2,I3
T700	I1,I3
T800	I1,I2,I3,I5
T900	I1,I2,I3

Construct the FP-Tree

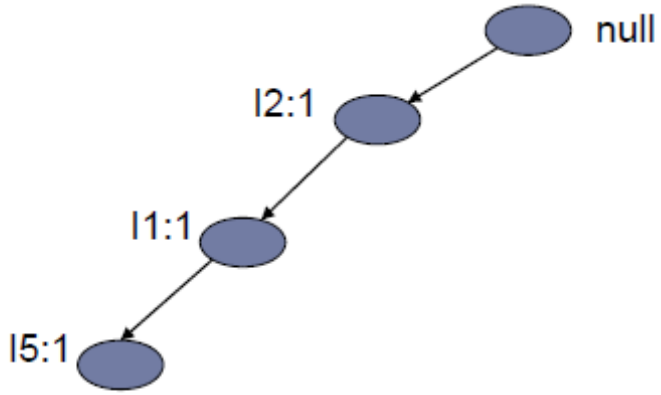
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

1- Order the items T100: {I2,I1,I5}
2- Construct the first branch: <I2:1>, <I1:1>,<I5:1>

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



Construct the FP-Tree

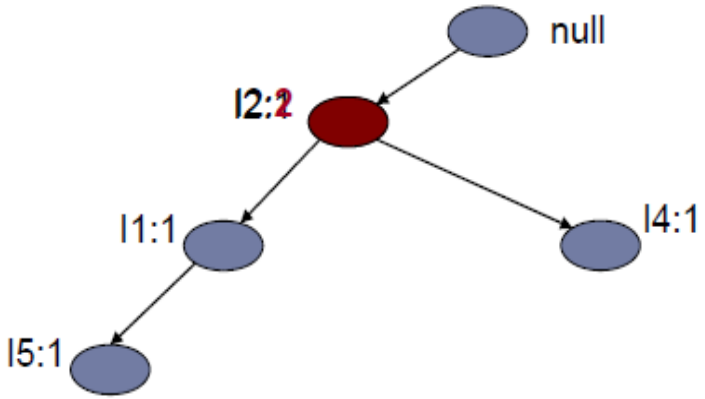
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

1- Order the items T200: {I2,I4}
2- Construct the second branch: <I2:1>, <I4:1>

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



Construct the FP-Tree

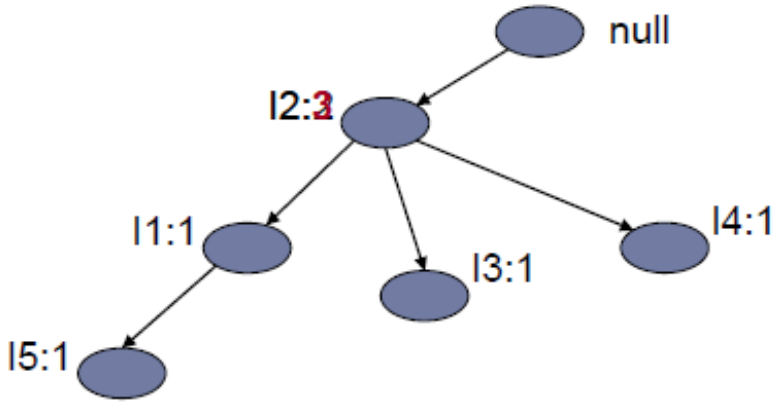
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

1- Order the items T300: {I2,I3}
2- Construct the third branch: <I2:2>, <I3:1>

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



Construct the FP-Tree

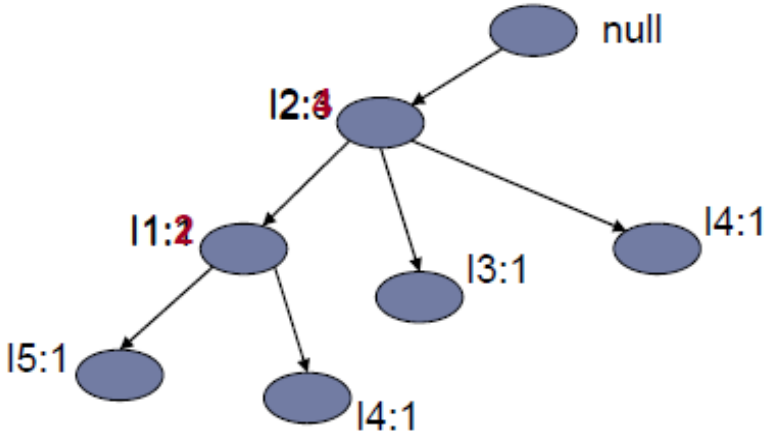
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

1- Order the items T400: {I2,I1,I4}
2- Construct the fourth branch: <I2:3>, <I1:1>,<I4:1>

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



Construct the FP-Tree

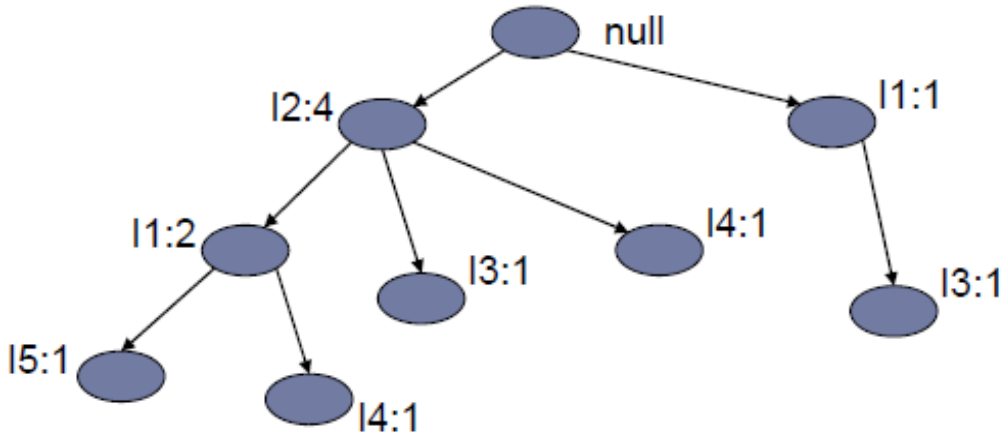
Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

- Create a branch for each transaction
- Items in each transaction are processed in order

1- Order the items T400: {I1,I3}
2- Construct the fifth branch: <I1:1>, <I3:1>

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2

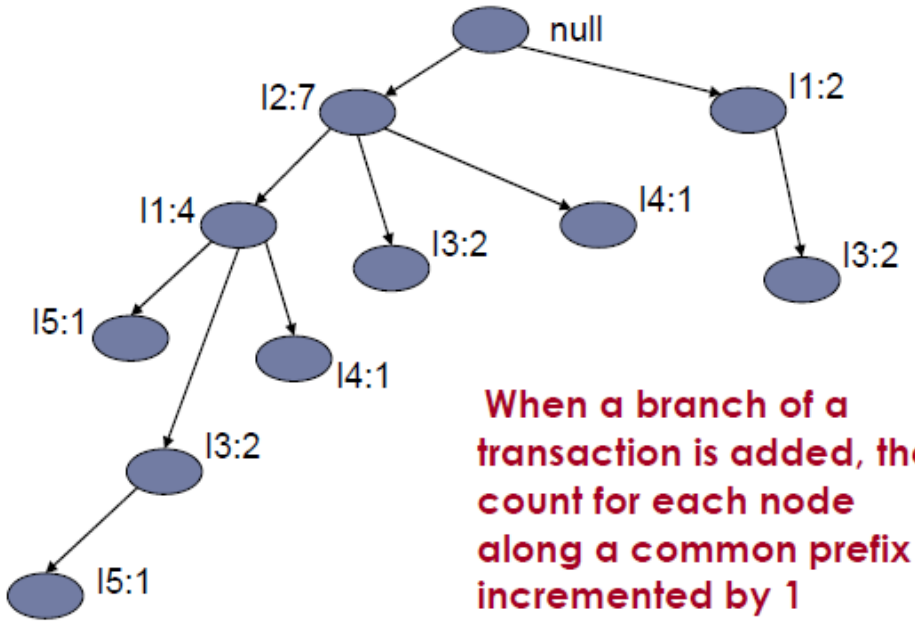


Construct the FP-Tree

Transactional Database

TID	Items	TID	Items	TID	Items
T100	I1,I2,I5	T400	I1,I2,I4	T700	I1,I3
T200	I2,I4	T500	I1,I3	T800	I1,I2,I3,I5
T300	I2,I3	T600	I2,I3	T900	I1,I2,I3

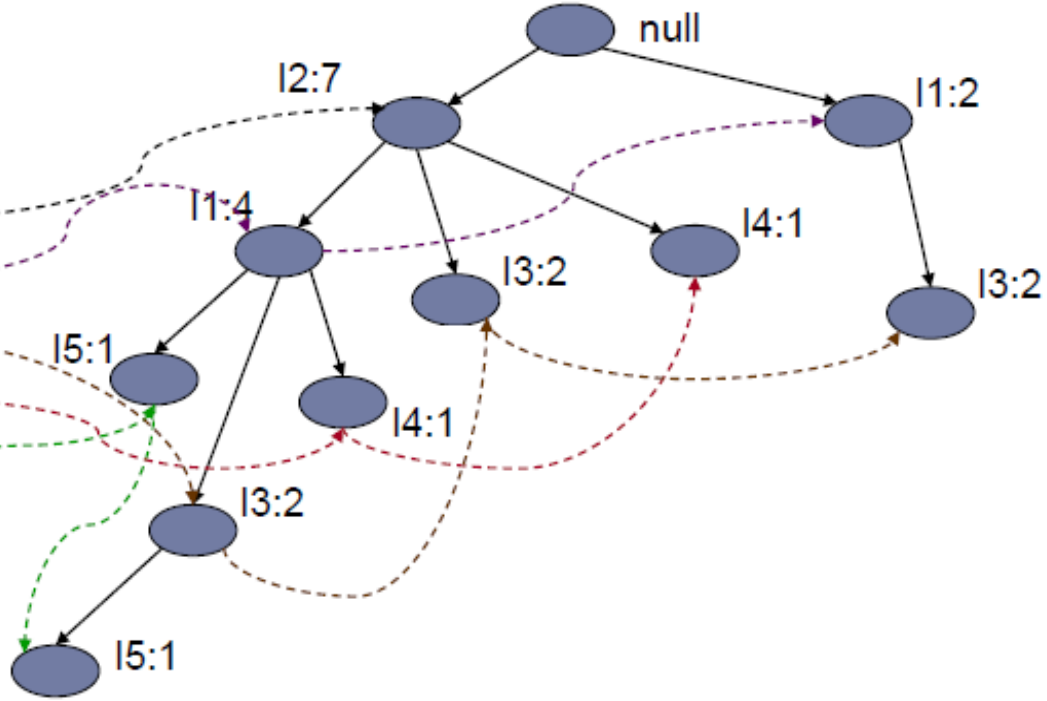
Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



When a branch of a transaction is added, the count for each node along a common prefix is incremented by 1

Construct the FP-Tree

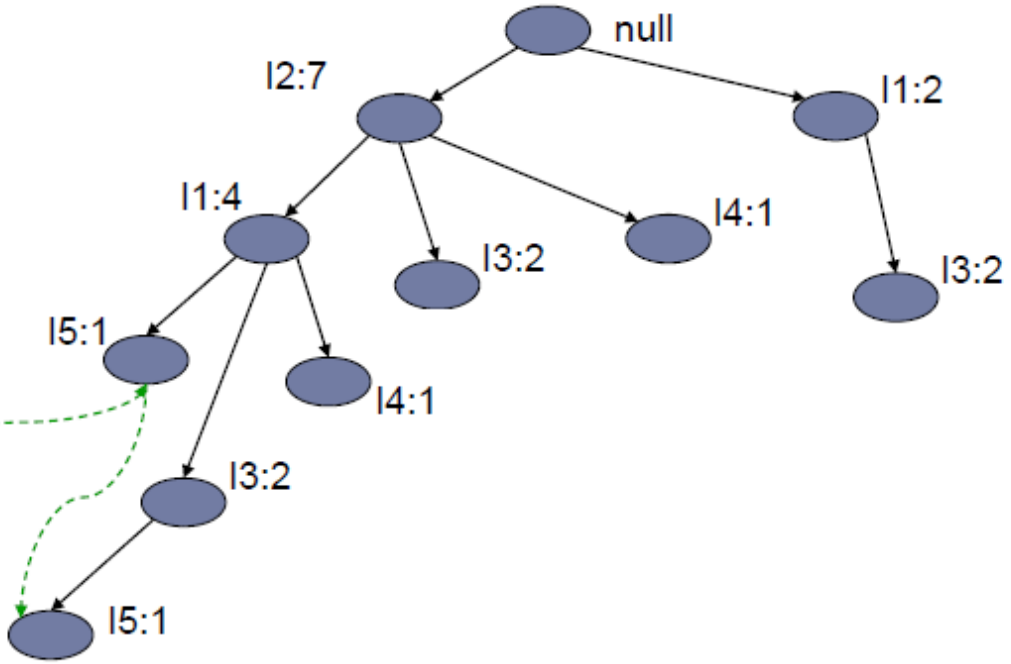
Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



The problem of mining frequent patterns in databases is transformed to that of mining the FP-tree

Construct the FP-Tree

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



-Occurrences of I5: <I2,I1,I5> and <I2,I1,I3,I5>

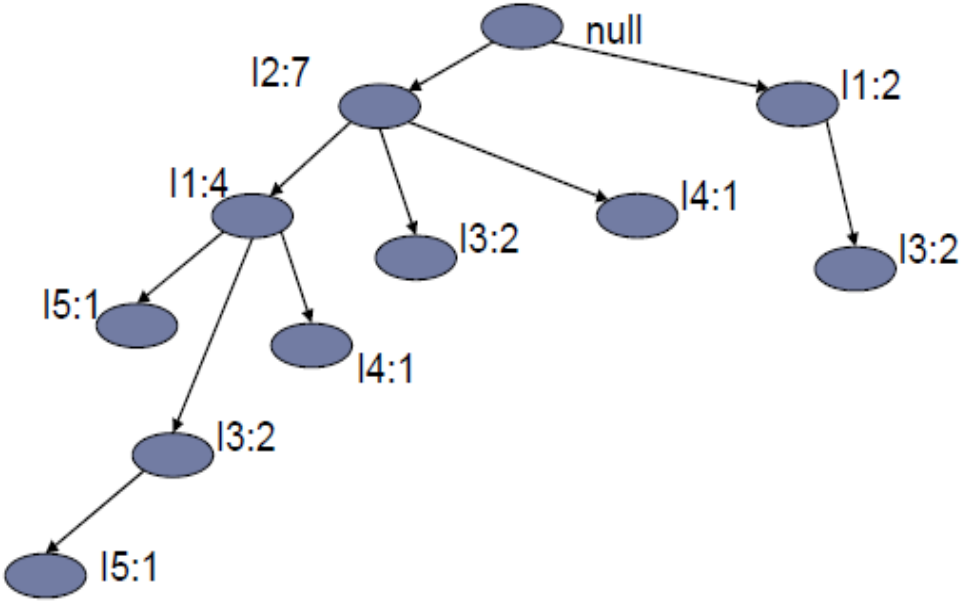
-Two prefix Paths <I2, I1: 1> and <I2,I1,I3: 1>

-Conditional FP tree contains only <I2: 2, I1: 2>, I3 is not considered because its support count of 1 is less than the minimum support count.

-Frequent patterns {I2,I5:2}, {I1,I5:2},{I2,I1,I5:2}

Construct the FP-Tree

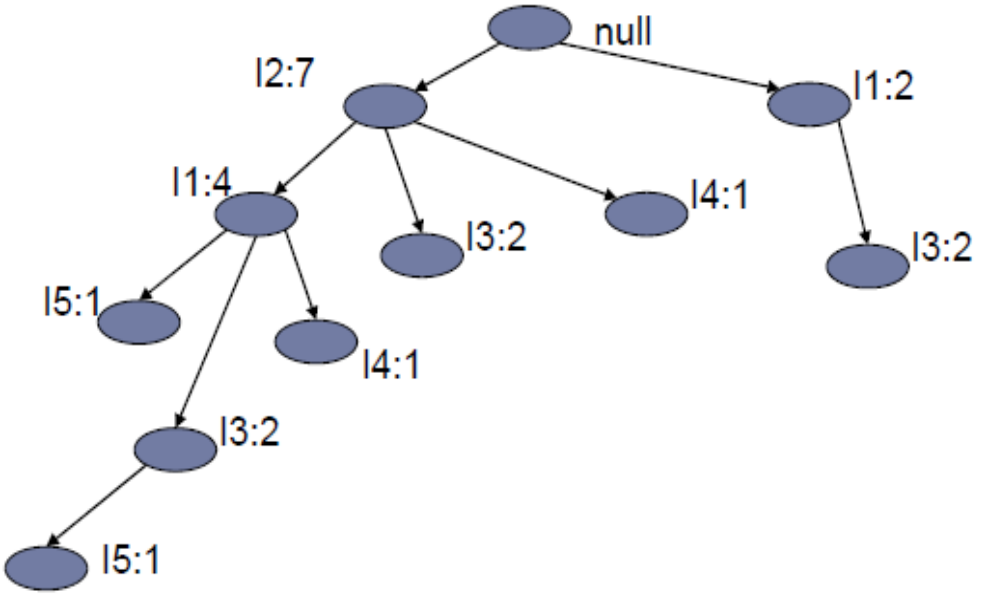
Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



TID	Conditional Pattern Base	Conditional FP-tree
I5	{{I2,I1:1},{I2,I1,I3:1}}	<I2:2,I1:2>
I4	{{I2,I1:1},{I2,1}}	<I2:2>
I3	{{I2,I1:2},{I2:2}, {I1:2}}	<I2:4,I1:2>,<I1:2>
I1	{I2,4}	<I2:4>

Construct the FP-Tree

Item ID	Support count
I2	7
I1	6
I3	6
I4	2
I5	2



TID	Conditional FP-tree	Frequent Patterns Generated
I5	<I2:2,I1:2>	{I2,I5:2}, {I1,I5:2},{I2,I1,I5:2}
I4	<I2:2>	{I2,I4:2}
I3	<I2:4,I1:2>,<I1:2>	{I2,I3:4},{I1,I3:4},{I2,I1,I3:2}
I1	<I2:4>	{I2,I1:4}

FP-growth properties

- FP-growth transforms the problem of finding long frequent patterns to searching for shorter once recursively and concatenating the suffix
- It uses the least frequent suffix offering a good selectivity
- It reduces the search cost
- If the tree does not fit into main memory, partition the database
- Efficient and scalable for mining both long and short frequent patterns

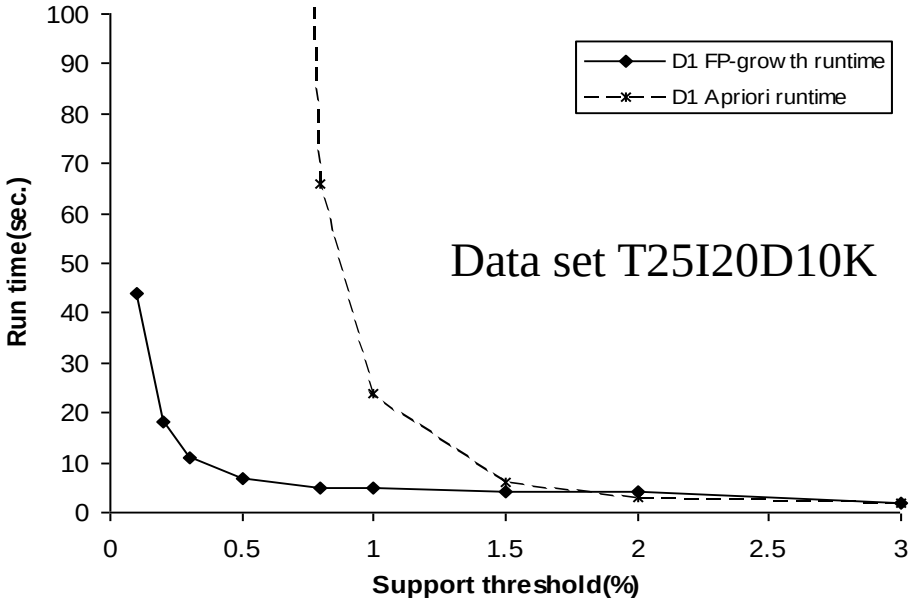
Benefits of the FP-tree Structure

- Completeness
  - Preserve complete information for frequent pattern mining
  - Never break a long pattern of any transaction
- Compactness
  - Reduce irrelevant info—infrequent items are gone
  - Items in frequency descending order: the more frequently occurring, the more likely to be shared
  - Never be larger than the original database (not count node-links and the *count* field)

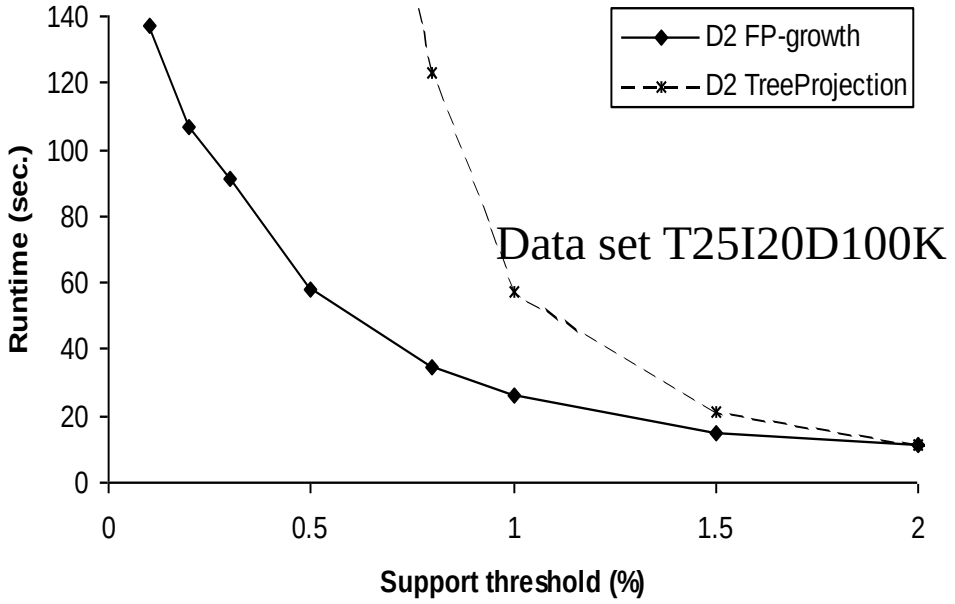
Scaling FP-growth by Database Projection

- What about if FP-tree cannot fit in memory?
  - DB projection
- First partition a database into a set of projected DBs
- Then construct and mine FP-tree for each projected DB
- Parallel projection vs. partition projection techniques
  - Parallel projection
    - Project the DB in parallel for each frequent item
    - Parallel projection is space costly
    - All the partitions can be processed in parallel
  - Partition projection
    - Partition the DB based on the ordered frequent items
    - Passing the unprocessed parts to the subsequent partitions

Performance of FP Growth in Large Datasets



FP-Growth vs. Apriori



FP-Growth vs. Tree-Projection

Advantages of the Pattern Growth Approach

- Divide-and-conquer:
  - Decompose both the mining task and DB according to the frequent patterns obtained so far
  - Lead to focused search of smaller databases
- Other factors
  - No candidate generation, no candidate test
  - Compressed database: FP-tree structure
  - No repeated scan of entire database
  - Basic ops: counting local freq items and building sub FP-tree, no pattern search and matching
- A good open-source implementation and refinement of FPGrowth
  - FPGrowth+ (Grahne and J. Zhu, FIMI'03)

Hash Based Algorithm

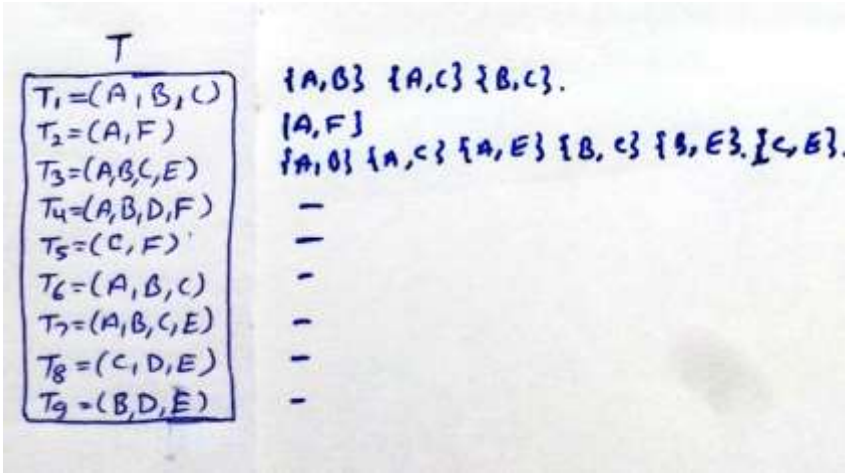
Similar to Apriori. Uses hash to reduce the size of set C's. This improve the computing efficiency while the number of candidate item sets to be checked is reduced.

Hash Based Algorithm DHP

- Similar to Apriori. Uses hash to reduce the size of set C's.
- This improve the computing efficiency while the number of candidate item sets to be checked is reduced.
- Assume a hash function $h(x, y) = (10 * ID(x) + ID(y)) \mod 7$

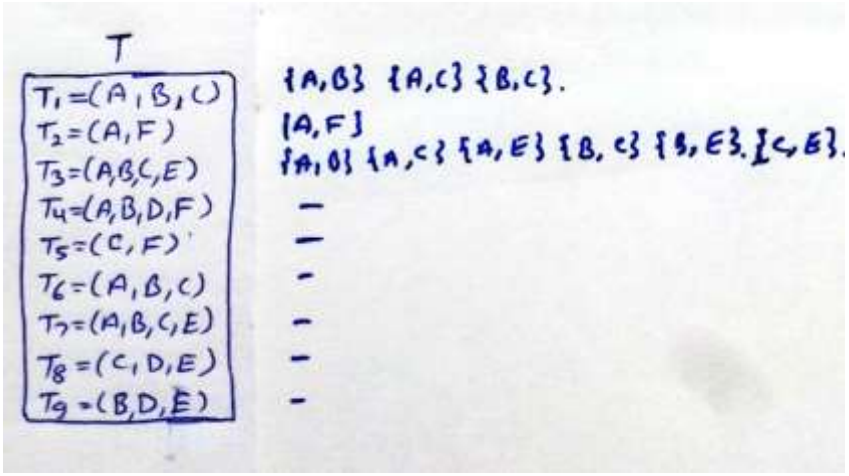
Hash Based Algorithm DHP

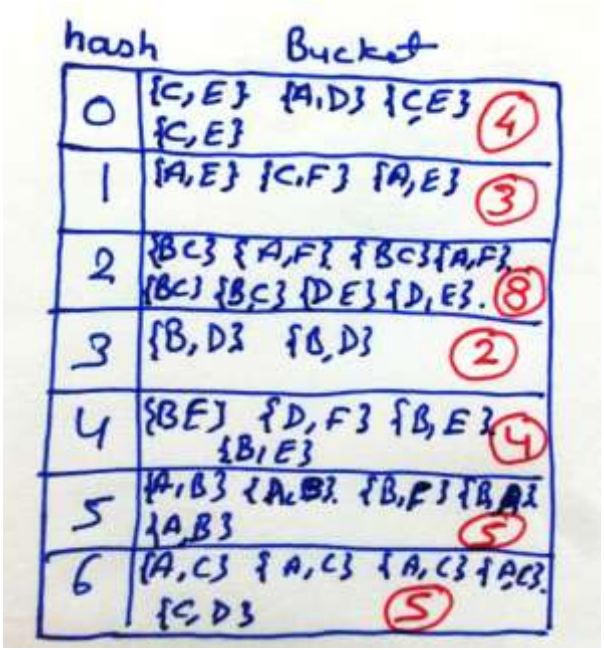
- Similar to Apriori. Uses hash to reduce the size of set C's.
- This improve the computing efficiency while the number of candidate item sets to be checked is reduced.
- Assume a hash function $h(x, y) = (10 * ID(x) + ID(y)) \bmod 7$



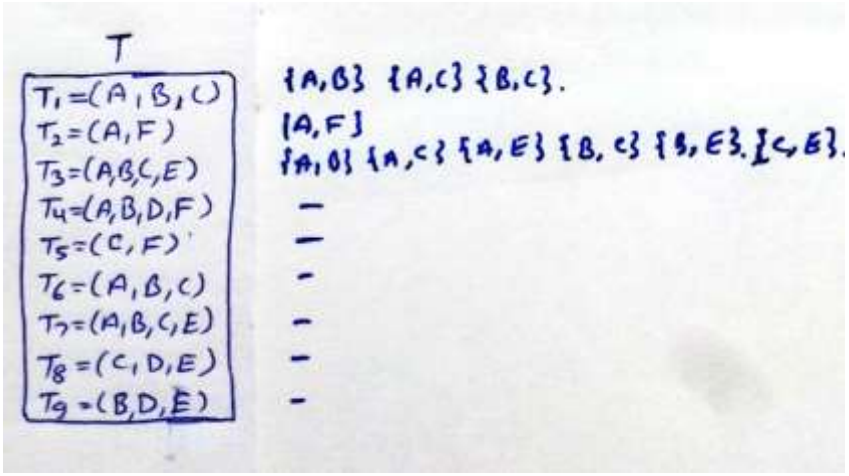
Hash Based Algorithm DHP

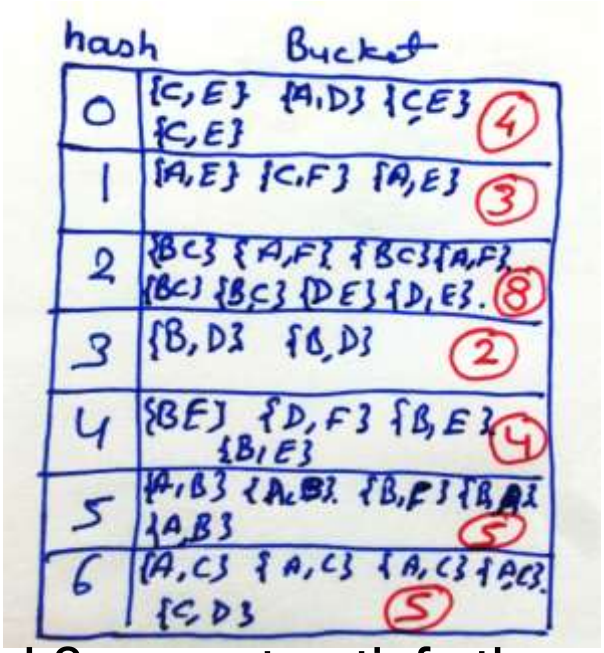
- Similar to Apriori. Uses hash to reduce the size of set C's.
- This improve the computing efficiency while the number of candidate item sets to be checked is reduced.
- Assume a hash function $h(x, y) = (10 * ID(x) + ID(y)) \bmod 7$





- Similar to Apriori. Uses hash to reduce the size of set C's.
- This improve the computing efficiency while the number of candidate item sets to be checked is reduced.
- Assume a hash function $h(x, y) = (10 * ID(x) + ID(y)) \bmod 7$





Note that the total counts for buckets 1 and 3 cannot satisfy the minimum support constraint, therefore, {A E}, should not be included in C_2.

Create hash table H_2 using hash function $h(x, y) = ((order\ of\ x) \times 10 + (order\ of\ y))\ mod\ 7$

H_2

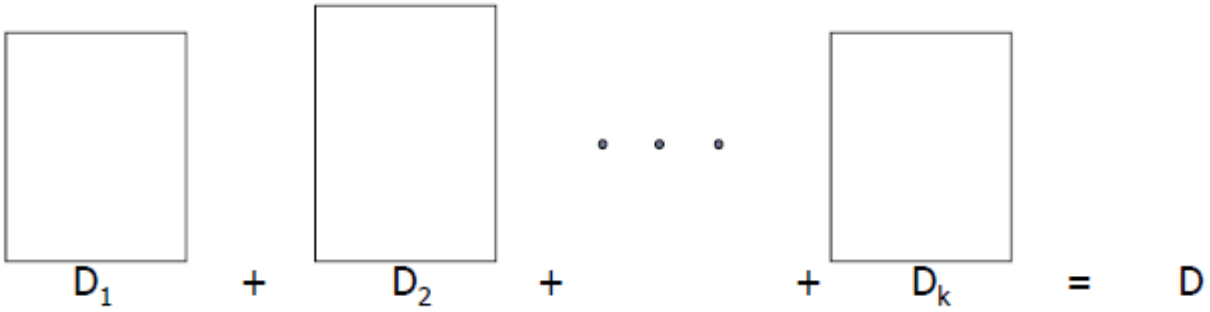
bucket address	0	1	2	3	4	5	6
bucket count	2	2	4	2	2	4	4
bucket contents	{I1, I4} {I3, I5}	{I1, I5} {I1, I5}	{I2, I3} {I2, I3} {I2, I3} {I2, I3}	{I2, I4} {I2, I4}	{I2, I5} {I2, I5}	{I1, I2} {I1, I2} {I1, I2} {I1, I2}	{I1, I3} {I1, I3} {I1, I3} {I1, I3}

If the minimum support count is, say, 3, then the itemsets in buckets 0, 1, 3, and 4 cannot be frequent and so they should not be included in C_2.

Hash-based technique (hashing itemsets into corresponding buckets): A hash-based technique can be used to reduce the size of the candidate k-itemsets, C_k, for $k > 1$. For example, when scanning each transaction in the database to generate the frequent 1-itemsets, L_1, we can generate all the 2-itemsets for each transaction, hash (i.e., map) them into the different *buckets* of a *hash table* structure, and increase the corresponding bucket counts A 2-itemset with a corresponding bucket count in the hash table that is below the support threshold cannot be frequent and thus should be removed from the candidate set. Such a hash-based technique may substantially reduce the number of candidate k-itemsets examined (especially when $k = 2$).

Partition: Scan Database Only Twice

- Subdivide the transactions of **D** into **k** non overlapping partitions
- Any itemset that is potentially frequent in **D** must be frequent in at least one of the partitions **Di**
- Each partition can fit into main memory, thus it is read only once
- **Steps:**
  - → Scan1: partition database and find local frequent patterns
  - → Scan2: consolidate global frequent patterns



Partition Based Algorithm

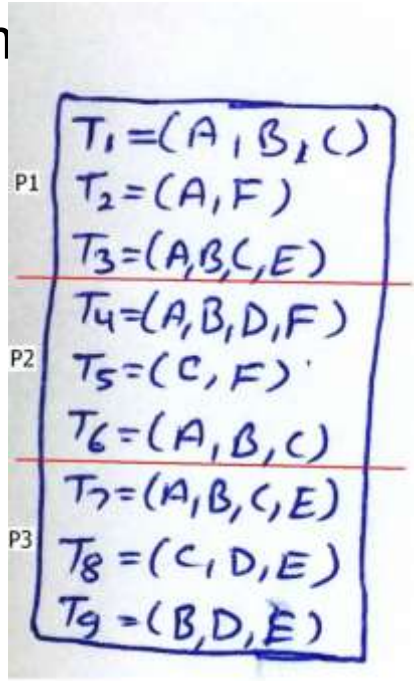
- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, ..., p_n$ must be a frequent item set in at least one of the n partitions

- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, ..., p_n$ must be a frequent item set in at least one of the n partitions
- It first scans partitions $p_1, p_2, ..., p_n$ to find local frequent item sets

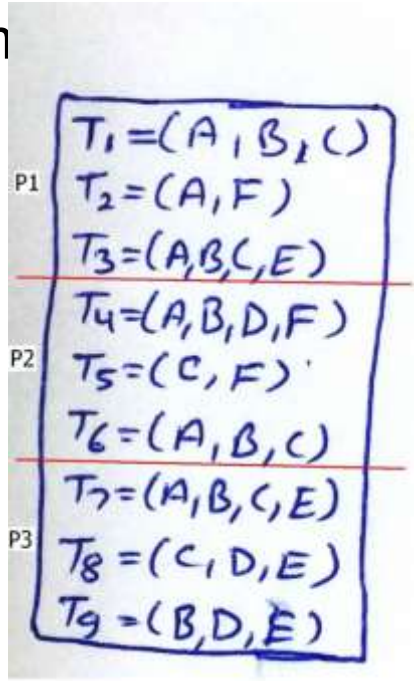
- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, \ldots, p_n$
- must be a frequent item set in at least one of the n partitions
- It first scans partitions $p_1, p_2, \ldots, p_n$ to find local frequent item sets Set of candidate item sets is constructed by taking the union

- Essentially based on a partition-based heuristic.
- A frequent item set in database D having n partitions $p_1, p_2, \ldots, p_n$
- must be a frequent item set in at least one of the n partitions
- It first scans partitions $p_1, p_2, \ldots, p_n$ to find local frequent item
- sets Set of candidate item sets is constructed by taking the union Scans the D second time to calculate the support of each item set

- Essentially based on a partition-based heuristic.
- A frequent item set in database *D* having *n* partitions $p_1, p_2, \ldots, p_n$
- must be a frequent item set in at least one of the *n* partitions
- It first scans partitions $p_1, p_2, \ldots, p_n$ to find local frequent item
- sets Set of candidate item sets is constructed by taking the union Scans the *D* second time to calculate the support of each item

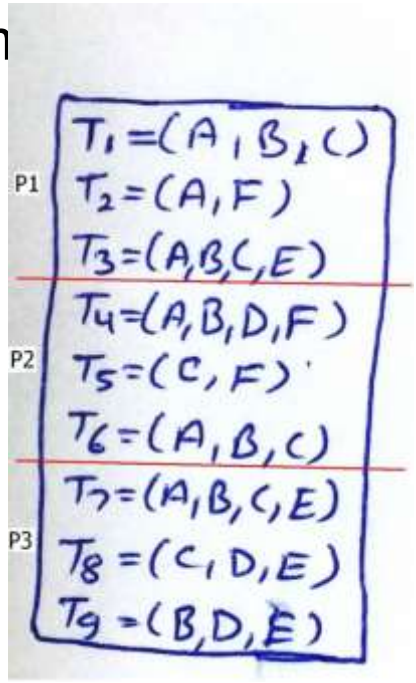


- Essentially based on a partition-based heuristic.
- A frequent item set in database *D* having *n* partitions $p_1, p_2, \ldots, p_n$
- must be a frequent item set in at least one of the *n* partitions
- It first scans partitions $p_1, p_2, \ldots, p_n$ to find local frequent item
- sets Set of candidate item sets is constructed by taking the union Scans the *D* second time to calculate the support of each item



Partition	Frequent Item set
P_1	{A} {B} {C} {AB} {AC} {BC} {ABC}
P_2	{A} {B} {C} {F} {AB}
P_3	{B} {C} {D} {E} {BE} {CE} {DE}

- Essentially based on a partition-based heuristic.
- A frequent item set in database *D* having *n* partitions $p_1, p_2, \ldots, p_n$
- must be a frequent item set in at least one of the *n* partitions
- It first scans partitions $p_1, p_2, \ldots, p_n$ to find local frequent item
- sets Set of candidate item sets is constructed by taking the union Scans the *D* second time to calculate the support of each item



Partition	Frequent item set
P_1	{A} {B} {C} {AB} {AC} {BC} {ABC}
P_2	{A} {B} {C} {F} {AB}
P_3	{B} {C} {D} {E} {BE} {CE} {DE}

Union is taken

Candidate item sets
{A} {B} {C} {D} {E} {F} {AB} {AC} {BC} {CE} {DE} {ABC} {ABC}

Second scan of D examines the required support